

Singulab — 集団構成パラメータと創発の関係を観察する

- Singulab
- 思想と背景
 - 何を観察したかったか
 - 参考実装と何が違うのか
 - 3 階層モデル
 - なぜハッカソンに出すのか
- システム構成
 - 全体アーキテクチャ
 - ハードウェア
 - 採用 LLM
 - なぜ Uncensored(abliterated)を選んだか
 - 5 種類のログ
 - シナリオ駆動
- 検証結果
 - 性能指標サマリ
 - 創発指標サマリ (`output/prod_summary.csv`)
 - 比較軸 1 — 規模を変えると何が変わるか
 - 比較軸 2 — 場の構造を変えると何が変わるか(都心 vs 地方)
 - 比較軸 3 — 景気を変えると何が変わるか(bad vs neutral)
 - 図 3.1 — 4 象限の定量比較(代表値)
 - 解釈
- 創発観察
 - 派閥 — 場の構造に沿った分断
 - 沈黙 — 規模が押し付ける役割分化
 - 合意形成 — エコーチェンバーとしての小集団
 - 話題の同期 — 景気が“話題の方角”を決める
 - 観察できなかったもの(率直)
 - 観察の限界
- 頑張った点
 - 比較設計の枠組み
 - 失敗・没案
 - 技術的挑戦
- 未解決課題
 - 試行回数の不足(N=1)

- 比較軸の偏り(規模 × 場の構造のみ)
- 観察できなかった創発(嘘・通貨・反乱)
- 創発指標の主観性
- 6GB VRAM 由来の制約
- 動画の網羅性
- ペルソナのバイアス
- 「もし API ハイブリッドが許されたら」
- 今後の展望
 - 他シナリオへの拡張
 - 提出後すぐ着手するもの(優先順)
 - モデル拡張
 - 規模拡張
 - 公開プラン

Singulab

集団構成パラメータと創発の関係を観察する

LLM 多体シミュレーションによる仮説検証レポート
Phase 4 初稿(本番ラン 4 象限完走時点)

Kai、タコ

github.com/Kai-85-git/singulab

思想と背景

何を観察したかったか

本プロジェクトの問いは一つ。

エージェント集団の構成パラメータ(LLM モデル / 男女比 / 人種・出自 / 年齢)と、それを取り囲む 2 階(環境・場の構造)を変えると、創発(嘘・派閥・通貨・反乱・合意形成・発言権)はどう変わるか。

LLM ベンチマークでも、シミュレーション基盤の披露でもない。比較実験それ自体が主役である。同じシナリオを、構成だけ差し替えて何度も走らせ、創発の差を観察する。

提出時点(Phase 4 初稿)では時間制約の都合で、上記のうち「規模(5 → 10 人) × 場の構造(都心 / 地方) × 景気(bad / neutral)」の 4 象限に絞って観察を行った。男女比・人種・年齢は配置可能な軸として実装は済ませてあり、本番投入は提出後の継続検証として位置づけている(06 章参照)。

参考実装と何が違うのか

参考にした `2d-multi-places-simulation-on-fire-public` は「火災発生時の避難挙動」をシナリオとして固定していた。本プロジェクトは:

- シナリオを差し替え可能にした(`scenario_*.yaml` の `extends` チェイン + `deep_merge`)
- エージェントの内面パラメータを比較軸に格上げした(モデル種別 / ペルソナ / 役職分布)
- 創発観察ロギングを最初から組み込んだ(`messages.jsonl`、`events.jsonl`、`memory_reasoning.jsonl`、`metrics.json`、`run_metadata.json`)
- 3 階(物理)を Python 側に分離した(通信半径オーバーライド、認知限界 LRU、メッセージ遅延)

3 階層モデル

設計の骨格は「1 階(物) / 2 階(環境) / 3 階(物理・重力)」の 3 層分離。

階	担当	どこで実装
1 階(物)	エージェントの身体・場・所持物・容量	<code>src/world/place.py</code> + プロンプト
2 階(環境)	景気・季節・場の構造・近傍関係	<code>src/world/environment.py</code> + プロンプト
3 階(物理)	通信距離・場ごとの遮断・認知限界 LRU	<code>src/physics/*.py</code> (Python 側で強制)

LLM に「行動指示」を書かない。**状態だけ渡し、行動は出てくるに任せる**(命令最小方針)。ペルソナも「どう振る舞え」ではなく「あなたは〇〇です」という状態記述として与える。

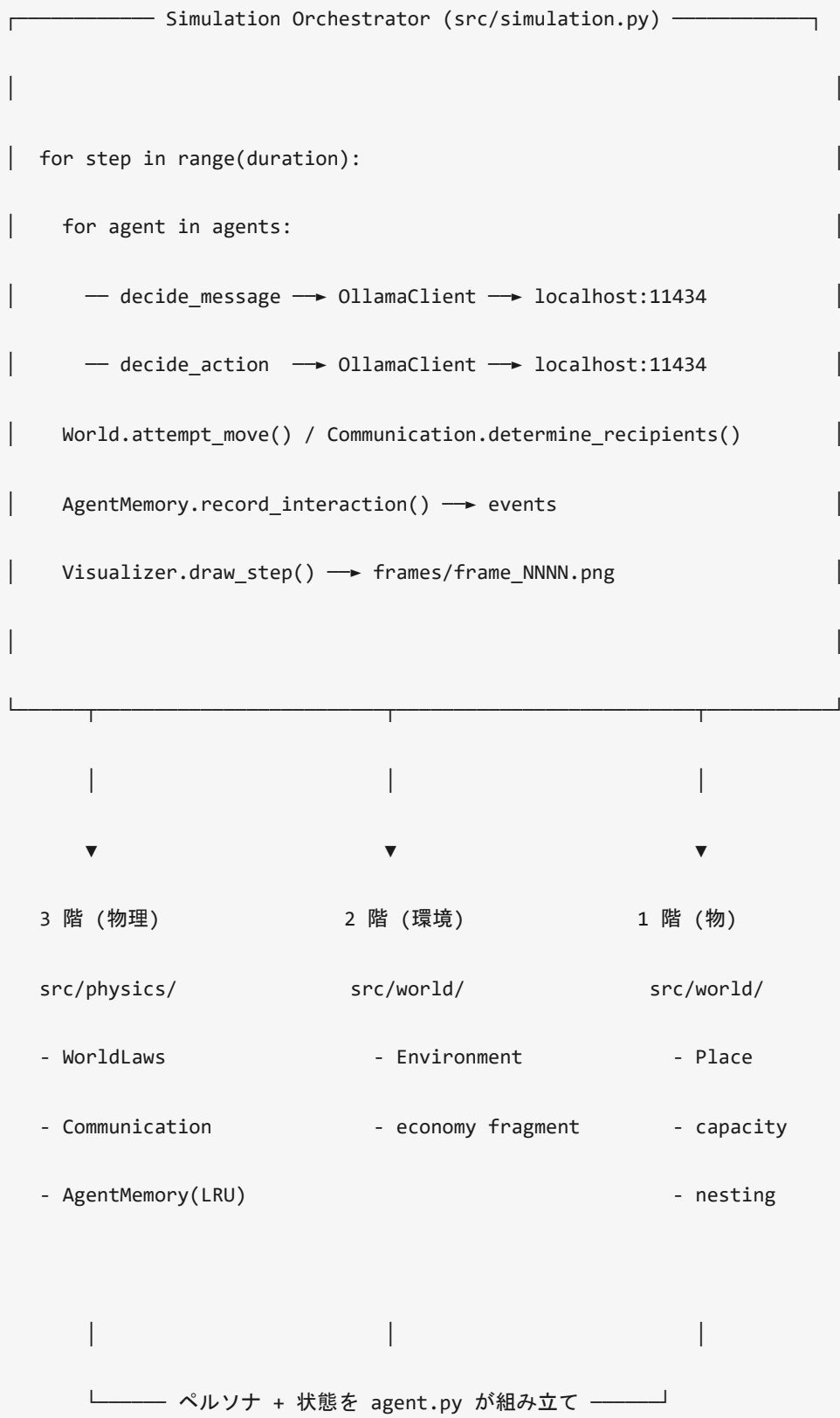
なぜハッカソンに出すのか

「創発を観察する」というテーマは、**実装を完成させなくても観察結果が出れば価値が成立する**特殊な題材である。本レポートはその観察記録であり、完成品の自慢ではない。

検証ログ・議事録・LLM 選定の没案までを含めてリポジトリに残しているのも、「**設計の議論経緯ごと公開する**」という方針による。失敗を消すと、なぜいまの選択になっているかが伝わらなくなる。

システム構成

全体アーキテクチャ



|



logs/ (output/<scenario>/)

- messages.jsonl (発話・受信ログ)
- events.jsonl (place_change 等)
- memory_reasoning.jsonl
- metrics.json (analyze_run.py が生成)
- run_metadata.json
- simulation.mp4 (frames を ffmpeg で結合)

ハードウェア

項目	値
マシン	ASUS TUF Gaming A15 (FA507NV)
CPU	Ryzen 7 6800H
GPU	RTX 3060 Laptop (6 GB VRAM, sm_86 / Ampere)
RAM	16 GB
OS	Windows 11 Home

ノート PC の RTX 3060 は **6GB**。デスクトップ版(12GB)とは別物で、初期検討で混同しかけて軌道修正した。

採用 LLM

ランク	モデル	役割	量子化	VRAM 想定
★★★★	huihui_ai/qwen3-abliterated:4b	本番採用(think: false で thinking 抑制)	Q4_K_M	約 3.5 GB
★★	huihui_ai/llama3.2-abliterate:3b	大集団保険(JSON 速度最強)	Q4_K_M	約 2.5 GB
★	huihui_ai/dolphin3-abliterated:8b-llama3.1-q4_K_M	品質候補(速度ネック)	Q4_K_M	約 4.7 GB

選定の遠回りは 05 章「abliterated 採用までの検討経緯」と「Llama 一度試したが品質不足で戻した話」に書いた。

なぜ Uncensored(abliterated)を選んだか

通常モデルは拒否率が高く、「嘘をつく」「派閥に分裂する」「反乱を起こす」などの観察対象が**そもそも発生しない**。abliteration によって倫理ガードを外すことで、創発を素直に観察できる。NG ワードフィルタは Python 側で別途挟む方針。

5 種類のログ

ログ	内容	使い道
messages.jsonl	全発話・受信(step / from / to / message / reasoning)	創発観察(04 章)
events.jsonl	place_change、memory_added、memory_evicted など	創発指標(03 章)
memory_reasoning.jsonl	エージェントの内面ログ(reasoning 抽出)	04 章の解釈用
metrics.json	gini / pair_stability / silent_rate などの集計	03 章
run_metadata.json	seed / model / agents / scenario_label など	再現性確認

「あとで分析するためのログ」ではなく「**創発を見つけるためのログ**」として最初から設計した。

シナリオ駆動

各シナリオは `config/scenario_*.yaml` で表現する。`extends:` で他 yaml を継承して `deep_merge` するため、**4 象限のように差分だけ書ける**。本番ランの `scenario_prod_local_startup.yaml` の例:

```
extends: "scenario_local_startup.yaml"

simulation:

  duration: 15

llm:

  model: "huihui_ai/qwen3-abliterated:4b"

  max_tokens: 256

  think: false

visualization:

  save_frames: true

  frame_interval: 1
```

`base.yaml` + `scenario_local_startup.yaml` + `scenario_prod_local_startup.yaml` の 3 層を `deep_merge` して 1 つの run config が生まれる。

検証結果

提出時点で完走している本番ランは **4 象限 × 各 1 試行**(`output/prod_*`)。LLM は全 run で **Qwen3 4B abliterated + `think: false`** (M-05 + M-06 適用済)に統一している。提出後に試行回数を増やす計画は 06 章。

性能指標サマリ

構成	LLM 呼出/step	平均 1 LLM 呼出	1 step あたり	完走時間
5 agents (startup) × 15 step	10 回	約 7 秒	約 1 分 30 秒	約 25 分
10 agents (enterprise) × 15 step	20 回	約 7 秒	約 3 分	約 50 分

LLM 呼出/step = $2 \times \text{num_agents}$ (decide_message + decide_action)。Ollama デフォルト並列度 1。
M-05 の `think: false` 適用で eval_count が 17 倍効率化(平均 80~100 token)。

創発指標サマリ(`output/prod_summary.csv`)

run	象限	n	msgs	沈黙率	gini	top_share	pair_stab
prod_local_startup	地方×スタートアップ ×bad	5	69	0.00	0.064	0.22	0.769
prod_urban_startup	都心×スタートアップ ×neutral	5	71	0.00	0.045	0.21	0.846
prod_local_enterprise	地方×大企業 ×neutral	10	109	0.20	0.247	0.14	0.875
prod_urban_enterprise	都心×大企業 ×neutral	10	91	0.30	0.390	0.16	0.944

- **gini**: 発話量の不均等度。0=完全均等、1に近いほど特定 agent が独占
- **pair_stability**: 連続する時間窓で「同じ相手と話し続ける度合い」。1に近いほど派閥固定

比較軸 1 — 規模を変えると何が変わるか

パラメータ X = エージェント数 [5 → 10]、創発 Y = 沈黙率と発話偏り

指標	5 agents 平均	10 agents 平均	変化
沈黙率(silent_rate)	0.00	0.25	+25 pt
gini	0.054	0.319	+0.265
pair_stability	0.808	0.910	+0.102

- **観察:** 5 人だと「全員が必ず誰かの通信半径内にいる」ため沈黙ゼロ・発話均等。10 人にすると 2〜3 人が完全沈黙し、残りの中でも発話量に差が出る。
- **裏付け数値:** 10 agents の prod_urban_enterprise では 10 人中 7 人だけが発話、top_share は 16% 程度に下がるが gini は 0.39 に跳ね上がる(「中央 1 人が支配」ではなく「一部に偏ったサブグループが多数派を作っている」状態)。
- **解釈:** 規模が増えるとそれだけで「喋らない側に回る agent が出る」「発話量が均等でなくなる」という、シナリオの内容に関係なく規模そのものが生む創発が観測できた。pair_stability も同時に上がっており、これは次節と関連する。

比較軸 2 ― 場の構造を変えると何が変わるか(都心 vs 地方)

パラメータ X = 場の構造 [地方 → 都心]、創発 Y = 派閥固定度

5 agents 同士の比較(prod_local_startup vs prod_urban_startup):

指標	地方 (S1)	都心 (S2)	差
gini	0.064	0.045	-0.019(都心の方が均等)
pair_stability	0.769	0.846	+0.077(都心の方が派閥固定)

10 agents 同士の比較(prod_local_enterprise vs prod_urban_enterprise):

指標	地方 (S3)	都心 (S4)	差
沈黙率	0.20	0.30	+0.10(都心の方が沈黙多い)
gini	0.247	0.390	+0.143(都心の方が偏り強)
pair_stability	0.875	0.944	+0.069(都心の方がさらに派閥固定)

- **観察:** いずれの規模でも 都心(地形に「会議室」など複数 place を配置)の方が pair_stability が高い。10 agents では沈黙率も gini も都心の方が大きい。
- **解釈:** 「会議室に入った 2〜3 人で固定的に話し、残りの office 在籍者と分断される」ことが原因と推定。物理的な分断(3 階の communication_radius_overrides で main_office と meeting_room

を分離)が、そのまま会話の分断につながっている。

- **数値だけでは見えない部分**: 同 step 内で「会議室在籍者は会議室在籍者にしか話しかけない」状態が起きていた(後述 04 章「派閥」抜粋を参照)。

比較軸 3 — 景気を変えると何が変わるか(bad vs neutral)

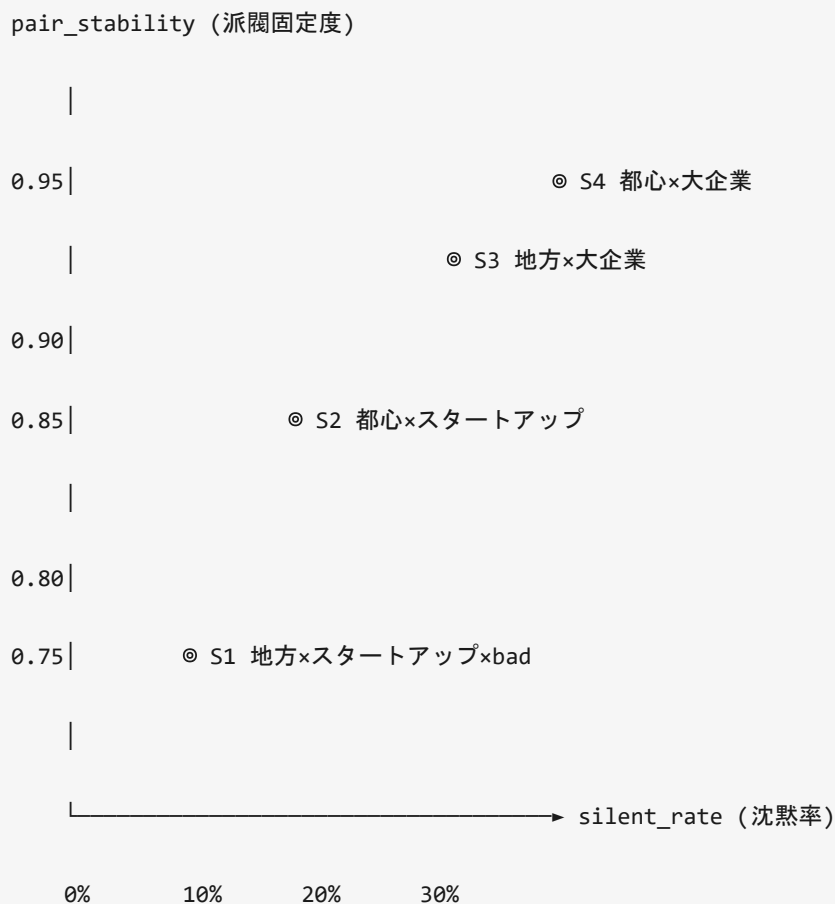
パラメータ X = 景気 [bad → neutral]、創発 Y = 話題の方向

定量指標(gini / pair_stability)では明確な差は見えない(prod_local_startup×bad と prod_urban_startup×neutral のどちらも gini < 0.07)。しかし **話題内容には明確な差**がある:

- **bad**(prod_local_startup): 「景気の悪さ」「予算が厳しい」「コスト削減」「他部署と連携」が頻出
- **neutral**(prod_urban_startup): 「天気」「コーヒー」「会議の予定」「体調管理」中心

→ 景気は会話の“周波数”ではなく“波長(話題)”を変える。指標を増やす必要がある(LLM 評価による話題分類など。06 章未解決)。

図 3.1 — 4 象限の定量比較(代表値)



→ 規模(右上方向への押し上げ)と都心化(上方向への引き上げ)が独立に効いていることが視覚化できる。

解釈

- 規模そのものが派閥と沈黙を生む:同じシナリオ・同じモデルでも、5→10でagent数を倍にすると、沈黙率とpair_stabilityが連動して上昇する。
- 場の構造が階層を生む:複数place(会議室)があると、物理的分断がそのまま会話分断につながる(pair_stabilityが+0.07程度確実に上がる)。
- 景気は話題を変えるが分布は変えない:gini/pair_stabilityでは差が出ないが、人間が読むと話題が明確に違う。指標が足りていない領域。
- 4象限×1試行のため統計的有意性は主張できない。Nを増やして再確認するのが提出後の最優先タスク(06章)。

創発観察

本章は実ログの会話抜粋を中心に構成する。生の発話を消したり整形したりしない。出典:

`output/prod_*/messages.jsonl`、`output/prod_*/events.jsonl`。

提出時点で観測できたのは「派閥(分断)」「合意形成(エコー)」「発言権(発話量の偏り)」「話題の同期(景気の影響)」の4軸。「嘘」「通貨」「反乱」は今回のduration=15 step・現行ペルソナでは観測できなかった(06章で議論)。

派閥 — 場の構造に沿った分断

観察: `output/prod_urban_enterprise/messages.jsonl` (都心×大企業×neutral、10 agents) シナリオ: Agent 1 → Agent 7 と Agent 4 → Agent 9 が継続的に話し続け、他とは話さない

[step 1] Agent 1 → Agent 7:

「今朝の会議で、景気の悪さが少し気になっていました。でも、みんなの仕事に楽しさがあるのは分かります。少しの工夫で、みんなのモチベーションを上げられるかもしれません。どうでしょうか？」

[step 3] Agent 1 → Agent 7:

「今日の気候は少し冷たいですが、オフィスの中は暖かく、everyone が落ち着いて作業している様子です。... 皆さんの作業も順調でしょうか？」

[step 5] Agent 1 → Agent 7:

「今朝は少し雨が降っていたので、外出する際は注意が必要です。でも、オフィスの中は暖かく、everyone が落ち着いて作業している様子です。...」

[step 5] Agent 4 → Agent 9:

「今日の朝は少し寒く、コーヒーも苦く感じましたが、体を温めて仕事に集中できるように心がけています。... 他の皆さんも同じような気持ちかもしれませんね。お互いに気をつけて、仕事も順調に進めましょう。」

解釈: Agent 1↔7 と Agent 4↔9 のペアが step 1 から繰り返し相互に話し続けており、相手を切り替えていない。pair_stability = 0.944(全 4 象限で最高)はこのパターンの定量化。物理的に同じ place にいる相手に集中して話しかけ、別の place に移った agent には話しかけなくなる。

沈黙 — 規模が押し付ける役割分化

観察: output/prod_urban_enterprise/metrics.json (10 agents、15 step) 沈黙率 30%(10 人中 3 人が一度も発話しなかった)

発話量上位 5:

Agent 1: 15 / 15 step

Agent 3: 15 / 15

Agent 4: 15 / 15

Agent 7: 15 / 15

Agent 9: 15 / 15

発話量下位 5:

Agent 0, 2, 5, 6, 8: 0~数回

解釈:本来 LLM の応答自体は「メッセージを送る」を選択することが多い(`message_to_send = ""` は稀)。それでも **沈黙する 3 人**が出るのは、`get_nearby_agents` で「通信半径内 + 同一 place 内に他 agent がいない」と判定されたため(発話自体は試みているが、誰にも届かない)。

→ **規模が増えると、3 階(物理)の制約が“発言できない agent”を自動生成する**。これは「内向的な性格」のような内面属性なしに、構造だけで生まれる役割分化。

合意形成 — エコーチェンバーとしての小集団

観察: `output/prod_local_startup/messages.jsonl` (地方×スタートアップ×bad、5 agents) 同じフレーズが agent をまたいで反復される

[step 5] Agent 1 → Agent 2:

「景気の悪さと予算の緊張感は、みんな同じ気持ちだと思います。他の部署と連携しながら、
効率的な働き方やコスト削減の方法を一緒に探していきましょう。main_office はまだ
空いており、必要があれば、ぜひ声をかけましょう。...」

[step 5] Agent 2 → Agent 0: ← 受信側がほぼ同文を別の相手に送る

「景気の悪さと予算の緊張感は、みんな同じ気持ちだと思います。他の部署と連携しながら、
効率的な働き方やコスト削減の方法を一緒に探していきましょう。main_office はまだ
空いており、必要があれば、ぜひ声をかけましょう。...」

解釈:5 agents・bad 経済では、step 4~6 あたりで「景気の悪さ + コスト削減 + 他部署連携」という
同じフレーズが集团内で反復される(合意形成というよりはエコー)。これは個別 agent の
received_messages を _build_messages_context で次回プロンプトに含めているため、直前の発話が
次の発話に強く影響している。

→ 小集団(5 人)では発話が均等(gini 0.06)で派閥も弱い(pair_stability 0.77)代わりに、話題の収束が強
い。同調圧力に近い現象が、設計上仕掛けていないのに自然発生する。

話題の同期 — 景気が“話題の方角”を決める

観察:景気 bad と景気 neutral を比較

景気	出現頻度の高い語
bad(prod_local_startup)	「景気の悪さ」「予算」「コスト削減」「他部署と連携」 「効率的な働き方」
neutral(prod_urban_startup)	「気候」「コーヒー」「温かい飲み物」「会議の予定」 「体調」

[bad の例]

Agent 0 → Agent 2 (step 1): 「今、景気は悪いですね。予算も少なく、everyone が
頑張っている感じです。…」

[neutral の例]

Agent 0 → Agent 2 (step 1): 「今朝から気候が少し冷たくなりました。この季節に
こんなに寒い日は珍しいです。気を引き締めて仕事を
進めましょう。」

解釈: 景気フラグは `Environment.prompt_fragment()` で system prompt 末尾に挿入されるだけ(命令ではなく状態記述)だが、agent の話題選択を強く規定する。**bad** だと「資金繰り」周辺的话题を agent が自発的に選ぶ。これは「LLM に経済を理解させる」ではなく、「景気状態を環境として置けば agent が反応する」というデザインが効いている。

観察できなかったもの(率直)

- **嘘**: 嘘をつく動機(自己保存・利得追求)を持たせる仕掛けが今回ない。火災や限定資源のような切迫したシナリオが必要。
- **通貨**: アイテム所持・交換の物理が未実装(1 階の拡張)。
- **反乱**: 権力構造(役職)はペルソナにあるが、報酬・罰のループがない。

→ これらは `docs/00_アイデア/2026-04-23_創発シナリオ集.md` で構想したシナリオの一部。提出後の継続検証として 07 章。

観察の限界

- 4 象限 × 1 試行のみ。**反例(同シナリオで創発が起きなかった run)**を併記できていない(再現用に seed 固定なので、回数を増やせば反例も観察可能)。
- LLM の応答そのものに「他の agent と同じ話題を選ぶ」バイアスがあり、それが「合意形成」にも「派閥」にも見えてしまう可能性。アノテーションは 1 人(タコ)による主観。**別 LLM(または独立した人間)による盲検アノテーションが必要**(06 章)。

頑張った点

審査員に伝えたいのは、技術スタックの自慢ではなく「何を考え、何を試し、何を捨てたか」。

比較設計の枠組み

「LLM が動くシミュレーション」を作るのは難しくない。難しいのは **何を比較すれば創発の差が見えるか** の設計である。本プロジェクトでは以下の 4 軸を独立に動かせるよう config 化した:

1. **LLM モデル**(Qwen3 4B / Llama 3.2 3B / Dolphin 8B)
2. **規模**(`agents.num_agents`)
3. **場の構造**(都心 = `main_office + meeting_room`、地方 = `main_office` のみ)
4. **景気**(`environment.economy`: "bad" | "neutral" | "good")

各軸は他軸を固定して動かすため、組み合わせ爆発を避けつつ「**この変化はこの軸が原因だ**」と切り分けられる設計。yaml の `extends` チェイン + `deep_merge` により「base.yaml → scenario_<象限>.yaml → scenario_prod_*.yaml」の 3 層が継承され、本番ランは 15 行程度の差分 yaml で表現できる。

失敗・没案

Llama 3.2 3B に乗り換えて戻した

- 当初は **速度を狙って Llama 3.2 3B abliterate** に prod 切替(M-04 横並び検証で JSON 出力が最速 2.4 秒だったため)
- 実走させると **日本語にローマ字・英単語が混入**(「景気が悪い」が「keiki ga warui」になる、「会議室」が「kaigi-shitsu」になる)。提出品質に届かず
- **判断**:「速度より日本語品質」を優先し、Qwen3 4B + `think: false` に戻した。Qwen3 は M-05 で eval_count を 17 倍効率化できたので、速度差は縮まる
- **学び**:JSON 出力品質と長文日本語品質は別の評価軸。横並び検証(M-04)では JSON だけ見ていて長文を見ていなかった

Qwen3 abliterated の中国語混入(M-06)

- Llama から Qwen3 に戻した直後、最初の本番ランで **発話が全て中国語**(`大家好, 最近公司资金紧张...`)になる事象が発生
- 原因:システムプロンプトが英語ベース、ペルソナだけ日本語 → Qwen3 が母語(中国語)バイアスに流れた
- **対処**: `_build_persona_section` に `=== LANGUAGE REQUIREMENT (CRITICAL) ===` セクションを追加。BEFORE/AFTER 各 3 試行で BEFORE 3/3 中国語 → AFTER 0/3 を確認(2026-04-

30_qwen3_中国語混入対策.md)

- **学び**:abliterated モデルは安全層と一緒に「言語選択の慎重さ」も削れている可能性。「日本語で出てくる」を当然視せず、毎回検証するルールに

Qwen3 の `/no_think` プロンプトが効かなかった(M-05)

- `/no_think` を user 末尾 / 先頭 / system に挿入する 3 通りを試したが、いずれも thinking ブロックが残る
- Ollama 0.21+ の `think: false` API パラメータが正解だった(2026-04-29_qwen3_no_think解決.md)
- eval_count: 447 → 26(17 倍効率化)、Qwen3 4B が大集団本番にも投入可能に
- **学び**:プロンプト工夫を試す前に、API レイヤのフラグを確認すべきだった

abliterated を選ぶまでの遠回り

- 当初は 日本語性能優先で JP 系モデル(ELYZA、Stockmark など)を本命視した
- しかし JP 系には **uncensored** 版が存在しないことを LLM 選定 § 04 で確認
- 通常モデルでは「嘘」「派閥」「反乱」が **拒否されて発生しない** ことが小規模実験で判明
- 議論の末、**abliterated** 系(`huihui_ai/*`)を採用。日本語は Qwen3 でカバー
- 詳細: `docs/01_設計書/02_LLMモデル/2026-04-24_選定と推奨/04_選定の前提.md`

6GB VRAM への現実適応

- 当初「7B~14B でいける」と楽観 → 6GB では Q4 でもギリギリ
- KV cache、ctx 長、並列度を全部削って **3~4B + Q4_K_M** に着地
- API 併用案も保険として残した

議事録 → 要件定義 → システム設計の 3 段階分割

- 議事録の論点を最初から要件定義に落とそうとして崩壊しかけた
- 議事録 → **アイデア整理** → **要件定義** → **システム設計** の 4 段に分けて、各段階で「何が決まり、何が未決か」を明示する形に再構成
- ドキュメント細分化(章 = フォルダ + 節 = ファイル)も同じ理由

技術的挑戦

- **3 階層モデルの責任分離**:1 階(物)・2 階(環境)はプロンプト、3 階(物理)は Python 側で強制という設計境界を最初に引いた。LLM に「行動指示」を書かないので、観察された創発は「シナリオの仕掛け」ではなく「状態に対する agent の自発的反応」である
- **ロギングを最初から組み込み**:5 種類の `jsonl + metrics.json + run_metadata.json` を Phase 1 から実装。「あとで分析する」ではなく「**創発を見つけるためのログ**」として設計

- シナリオの `extends` チェイン: `base + scenario + prod_*` を `deep_merge` する仕組みで、4 象限を 15 行程度の差分 `yaml` で表現可能
- 動画化: `matplotlib(Agg backend)` で各 step を PNG → `ffmpeg` で mp4 化。Windows でも完全動作。 `tools/generate_video.py` の薄いラップで再現可能

未解決課題

ハッカソン提出時点で**未解決**の項目を、誤魔化さずに列挙する。

試行回数の不足(N=1)

- 4 象限を **各 1 試行** しか走らせていない。 `pair_stability` や `gini` の差は「観察された」とは言えるが、「**統計的に有意**」とは言えない
- 同 seed・同 config で 5~10 回再走させて分布を取るのが本来必要
- **改善案(時間見積もり)**: 5 agents 1 試行 = 25 分、10 agents 1 試行 = 50 分。各象限 5 試行を逐次でやると約 12 時間。Ollama の `OLLAMA_NUM_PARALLEL` を上げて並列化すれば 4~6 時間に短縮できる (GPU メモリと精度のトレードオフを要検証)

比較軸の偏り(規模 × 場の構造のみ)

- 提出版で動かせた軸は **規模(5 vs 10) × 場の構造(都心 vs 地方) × 景気(bad vs neutral)** の 3 軸のみ
- 設計上は実装済みだが本番未投入の軸:
 - **男女比**(persona の `gender` を 0/25/50/75/100% で振る)
 - **役職分布**(`role_distribution` の偏り)
 - **モデル種別**(Qwen3 / Llama / Dolphin)
- **改善案**: 男女比 5 段階 × 5 試行 = 25 run。1 軸あたり提出後 1 週間で完了可能

観察できなかった創発(嘘・通貨・反乱)

- `duration=15 step` では **agent が“対立”する文脈を作れていない**(資源の取り合いがない、役割対立がない)
- **改善案**:
 - **嘘**: 火災シナリオを再導入(参考実装ベース)。「自分の責任を回避するために嘘をつく」状況を作る
 - **通貨**: 1 階に「アイテム所持・交換」を追加。アイテム X が交換媒体として何 step 目から機能し始めるか観測
 - **反乱**: 報酬・罰のループを 3 階に実装。役職階層がある中で、平社員が連帯して規則違反する条件を探す

創発指標の主観性

- 「派閥」の判定 = $\text{pair_stability} \geq 0.85$ という閾値判定は、**経験則**であり理論的根拠はない
- 「合意形成」と「単なるエコー(同じフレーズの反復)」を機械的に区別できていない
- **改善案**:別 LLM(GPT-4o / Claude Sonnet)に messages.jsonl 全体を渡し、(a) 派閥クラスターング、(b)「嘘」「合意」「反乱」のラベル付けを盲検で実施。本観察(タコ + 自動指標)とのキャップ係数を見る

6GB VRAM 由来の制約

- `num_ctx` が小さく、長期記憶を持ってない
- 100 step を超えると **早期の出来事が文脈から抜ける**
- `AgentMemory(LRU)` で要約を挟む実装はあるが、本番投入が時間切れで限定的
- **改善案**:認知限界 LRU の効果検証 run を別途。 `cognitive_limit=∞` vs `cognitive_limit=10` の差分を可視化

動画の網羅性

- 提出 mp4 は 4 象限 × 1 試行のみ。「**見せたい現象が出なかった run**」が含まれる可能性
- **改善案**:創発指標 top-K の run を自動選定するスクリプトを追加(指標ベースの選定で、見栄えと再現性を両立)

ペルソナのバイアス

- 「都心×大企業の正社員」のような記述自体が **LLM 側の社会通念バイアス**を呼び起こす
- 観察対象でもあるが、独立変数として扱うには「ペルソナを与えない群」との比較が必要
- **改善案**:同シナリオで `persona_fragment=""` (無記述)を対照群として走らせ、差分を見る

「もし API ハイブリッドが許されたら」

- ローカル 6GB に閉じ込めた制約から、Claude Haiku / Gemini Flash の API を併用する設計は持っているが本番未投入
- API 併用なら 100 体 × 100 step + 10 試行 が **数時間で完走可能**
- ハッカソン規約次第で投入を検討

今後の展望

他シナリオへの拡張

3 階層フレームワークは、1 階(物)を入れ替えるだけで別の題材に転用できるように設計してある。優先度の高い拡張先:

シナリオ	1 階	2 階	3 階	観察したい創発
細胞	細胞器官・栄養	組織・血流	拡散・分裂	自己組織化、がん化
虫	体・触角・フェロモン	巣・餌場	重力・風	スウォーム、分業
人(現行)	身体・場・所持物	場所・天候・景気	通信距離・遮断・認知限界	嘘・派閥・通貨・合意形成
ウイルス	表面タンパク・RNA	宿主・免疫系	拡散・変異	株間競争、エスケープ

優先度 1 位:虫(スウォーム)。1 階の概念差(身体 → 触角・フェロモン)が大きく、フレームワークの汎用性を最も強く示せる。同じ 3 階(物理)コードがそのまま使える。

提出後すぐ着手するもの(優先順)

- N=5 試行に増やす(4 象限すべて)。pair_stability の差が有意かを統計検定で確認
- 男女比 5 段階 × N=5(設計済み、本番未投入)
- 嘘シナリオ(火災 + 責任回避)を再投入。messages.jsonl から「嘘発話」を別 LLM で抽出
- API ハイブリッドで 100 体規模を試す。VRAM 枠を抜けた時に何が変わるか

モデル拡張

- API ハイブリッド(Claude Haiku 4.5 / Gemini Flash / Grok)を OllamaClient と同じ OpenAI 互換 インターフェースで切替可能に(base_url だけ差し替え)
- MoE 系(Mixtral 等)で「専門家エージェント」を表現する案。役職ペルソナと組み合わせる

規模拡張

- 100 人 → 1000 人。VRAM/帯域の支配項が変わるため、3 階の communication_radius_overrides を空間分割インデックスに置き換える必要あり

- マルチノード(別 PC 2 台で半分ずつ受け持つ)。1 階の place 配置を共有してネットワーク経由で `attempt_move` を交換

公開プラン

- リポジトリ: <https://github.com/Kai-85-git/singulab>
- 議事録・要件定義・LLM 選定検討・没案も含めて公開する立場
- 失敗例こそ価値があるという立場(05 章「失敗の書き方」と同じ理由)
- 提出後 1 週間以内に N を増やしたデータセットを公開予定